

Building Numo for \$0 on Windows

How to build and test the iOS app without a Mac or a paid Apple account

Yes. If your requirement is **\$0 total cost**, then I would change the approach.

The important correction is: **you do not need live Xcode debugging to build Numo for personal use**. You need a way to iterate quickly enough from Windows.

Apple officially supports free Personal Team provisioning for installing apps on your own devices. The current limits are 10 App IDs, 3 devices, 3 apps per device, and provisioning expires after 7 days. ([Apple Developer](#))

The fully free setup

```
Windows PC
├── VS Code
├── WSL
│   ├── Swift source
│   ├── Hono
│   └── Supabase CLI
├── GitHub
│   └── GitHub Actions → macOS runner
├── iPhone
│   └── Sideloadly / libimobiledevice
```

You lose the **Xcode debugger**, but you can still get a surprisingly usable development loop.

Replace live debugging with 4 things

1. Build frequently with GitHub Actions

Instead of:

edit → commit → build → download → install

make the process almost automatic:

```
Save code
↓
git push
↓
GitHub Actions
↓
macOS runner
↓
build Numo
↓
IPA artifact
```

You then download/install the latest build.

For a personal app, this is acceptable during early development.

2. Add an in-app debug screen

This is the part I would strongly recommend for Numo.

Create:

```
Settings
→ Developer
→ Debug
```

Show things such as:

```
Numo Debug

App version
Build number
Database status
Sync status
Last sync
Pending sync operations

Budget
1500 DT

Spent
284 DT

Remaining
1216 DT

Today's allowance
60.80 DT

Last expense
12 DT
Food
12:04

Errors
0
```

Also add buttons:

```
[ Seed Test Data ]
[ Clear Local Data ]
[ Force Sync ]
[ Recalculate Budget ]
[ Export Debug Log ]
[ Simulate New Day ]
```

For your app, this is arguably more useful than a debugger for many bugs.

3. Use `print()` + device logs

Your app can produce logs such as:

```
print("[EXPENSE] Creating expense: \$(amount)")
print("[SYNC] Uploading expense: \$(expense.id)")
print("[BUDGET] Remaining: \$(remaining)")
```

`libimobiledevice` provides tools for communicating with iPhones, including debugging and debugserver proxy functionality. ([GitHub](#))

So your Windows/WSL environment can become:

```
idevice_id -l  
ideviceinfo  
idevicesyslog
```

You can inspect runtime logs without owning a Mac.

4. Use crash reports

You can also retrieve crash reports from the device using the same ecosystem. ([GitHub](#))

So a crash becomes:

```
iPhone  
↓  
crash report  
↓  
Windows  
↓  
inspect stack / logs  
↓  
fix  
↓  
GitHub Actions
```

What about actual breakpoints?

This is the one thing you **cannot reproduce properly for free from Windows**.

You cannot turn WSL into Xcode.

You cannot run the iOS Simulator in WSL.

You cannot get the normal:

```
Breakpoint  
↓  
pause  
↓  
inspect Swift variable  
↓  
step over  
↓  
step into
```

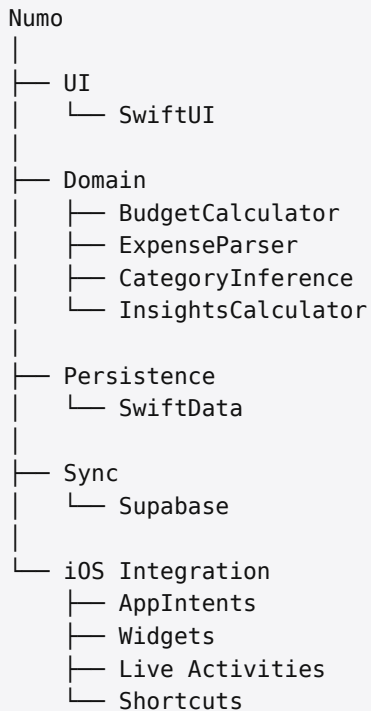
workflow without Apple's development environment.

`libimobiledevice` exposes lower-level debugging functionality, including `idevicedebugserverproxy`, but this is not equivalent to having Xcode's complete iOS development environment. ([GitHub](#))

But there is a better trick for Numo

Build Numo so **most of your business logic is independently testable**.

For example:



Then your critical calculations can be tested independently.

Example:

```
let budget = 1500
let spent = 284
let remaining = budget - spent

assert(remaining == 1216)
```

You can test a large portion of the app's logic without touching an iPhone.

And there is one major limitation you should accept

For Numo's **native Apple features**, you still need periodic physical-device testing:

- Widgets
- Shortcuts
- App Intents
- Dynamic Island
- Live Activities
- Action Button
- notifications
- background behavior
- SwiftData behavior on-device

You can build these through your cloud compilation workflow and test them on your iPhone.

The free Apple account is enough for personal-device testing, although the 7-day provisioning cycle means you'll need to reinstall periodically. ([Apple Developer](#))

So I would build Numo like this

\$0 stack:

Part	Tool
Editor	VS Code
OS environment	WSL
Swift source	Swift
UI	SwiftUI
Local DB	SwiftData
Backend	Hono
Cloud DB	Supabase
Git	GitHub
iOS compilation	GitHub Actions macOS runner
Device installation	Sideloadng
Device logs	libimobiledevice
Testing	XCTest + physical iPhone
Apple account	Free Personal Team

Apple confirms free registration and personal-device testing without paid Developer Program membership. ([Apple Developer](#))

The key change

Don't try to recreate **Xcode on Windows**.

Instead, make Numo **debuggable without Xcode**.

For a personal app, I'd build a serious internal **Developer Mode** into Numo from day one. That gives you visibility into budgets, database state, sync, App Intents, parsed expenses, errors, and background operations without paying for a Mac.

So yes, you can build Numo for \$0. The tradeoff is slower build/install cycles and no traditional interactive Xcode debugger, not an inability to develop the app.

Sources

1. Developer account overview - Basics - Account - Help - Apple Developer

<https://developer.apple.com/help/account/basics/about-your-developer-account>

2. GitHub - libimobiledevice/libimobiledevice: A cross-platform protocol library to communicate with iOS devices · GitHub

<https://github.com/libimobiledevice/libimobiledevice>